

บทที่ 11

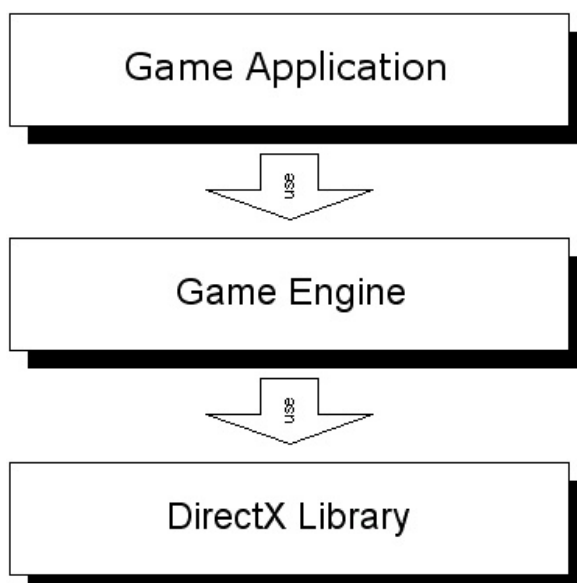
แนวความคิดและการออกแบบ

11.1 ความต้องการ (Requirement)

ในขั้นตอนแรกจะต้องกำหนดคุณลักษณะของแอปพลิเคชันที่จะสร้างขึ้น ซึ่งในโครงการนี้ได้กำหนดคุณสมบัติของแอปพลิเคชันขั้นต้นดังนี้

1. เป็นแอปพลิเคชันประเภทเกมที่แสดงผลด้วยกราฟิก 3 มิติ
2. กราฟิก 3 มิติที่แสดงเป็นตัวละครในเกมจะต้องมีความสามารถในการทำแอนิเมชัน
3. มีความสามารถในการตรวจสอบการชนของวัตถุเพื่อความจริง
4. เล่นเพลงจากไฟล์ในรูปแบบของเวฟในส่วนของเอฟเฟ็กต์และในรูปแบบ MP3 ในส่วนของเสียงเพลง
5. เล่นไฟล์วิดีโอได้
6. สามารถใช้งานอุปกรณ์อินพุตได้หลากหลายทั้งเมาส์, คีย์บอร์ด และจอยสติค

โครงการนี้ทำการพัฒนาแอปพลิเคชันประเภทเกมสับนไลบรารีของ DirectX แต่เนื่องจากโคเร็กเอ็กซ์ในขณะนี้ (เวอร์ชัน 8) ประกอบขึ้นด้วยอินเทอร์เฟซและเมทอดที่ทำงานในขั้นพื้นฐานเท่านั้น เช่น การสร้างบัฟเฟอร์สำหรับเก็บข้อมูลเวอร์เท็กซ์, การคัดลอกข้อมูลไปยังฟรอนต์บัฟเฟอร์ เป็นต้น ดังนั้นเพื่อที่การพัฒนาซอฟต์แวร์เกมจะมีความสะดวกรวดเร็วและมีความผิดพลาดน้อยลงแล้วจำเป็นที่จะต้องนำเอาอินเทอร์เฟซและเมทอดที่ทำงานพื้นฐานเหล่านี้มารวมกันสร้างเป็นคลาสเพื่อทำงานอย่างใดอย่างหนึ่งและทำให้อยู่ในรูปที่ใช้งานง่ายโดยจำกัดขอบเขตการเข้าถึงอินเทอร์เฟซและเมทอดและกำหนดวิธีการใช้งานบางอย่าง ซึ่งคลาสที่สร้างขึ้นในส่วนนี้เรียกว่า เกมเอนจิน (Game Engine) นั่นเอง หลังจากนั้นจึงใช้คลาสที่สร้างขึ้นมาใช้ในการพัฒนาแอปพลิเคชันเกม



รูปที่ 11-1 ลำดับชั้นในการพัฒนาเกม

นอกจากนั้น โมเดลและภาพ 2 มิติที่จะนำมาใช้ในการพัฒนาโปรแกรมก็มีความจำเป็นที่จะต้องกำหนดไว้ ซึ่งข้อมูลในไฟล์ที่ได้จากแอปพลิเคชันสร้างโมเดล 3 มิติ และไฟล์ 2 มิติที่มี มักจะไม่ตรงกับความต้องการของแอปพลิเคชันเกม

ความต้องการรูปแบบของไฟล์ที่เก็บ โมเดล 3 มิติที่ใช้ในแอปพลิเคชันเกมสำหรับโครงการนี้

1. มีข้อมูลเอนิเมชันและเก็บข้อมูลเอนิเมชันแต่ละท่วงท่าแยกจากกันโดยเด็ดขาดสามารถใช้งานได้ง่าย
2. ข้อมูลเอนิเมชันที่ต้องการจะต้องเป็นเอนิเมชันในรูปแบบที่ใช้กับอินเด็กซ์เวอร์เท็กซ์เบลนดิงได้ คือ จะต้องเป็นข้อมูลที่มีค่าน้ำหนัก, ค่าเมตริกซ์และค่าชื่อของโบนที่เกี่ยวข้อง
3. ข้อมูลเวอร์เท็กซ์ของโมเดลจะต้องอยู่ในรูปแบบที่สามารถโหลดเข้าไปใช้งานได้ทันทีโดยไม่ต้องมีการประมวลผลใดๆ เกิดขึ้นอีก
4. จะต้องเป็นไฟล์แบบไบนารีเนื่องจากไฟล์ในรูปแบบของเท็กซ์ไฟล์จะเสียสโหทัยในการแปลงจากสตริงเป็นข้อมูล
5. ข้อมูลเมตาที่เรียกเป็นข้อมูลในระดับโมเดลไม่ใช่ข้อมูลในระดับเฟรม

ความต้องการรูปแบบของไฟล์ที่เก็บข้อมูล 2 มิติที่ใช้ในแอปพลิเคชันเกมสำหรับโครงการนี้

1. จะต้องประกอบด้วยบิตข้อมูลที่แสดงค่าความโปร่งใส
2. อยู่ในรูปแบบที่สามารถเขียนข้อมูลจากไฟล์ลงบัฟเฟอร์ได้อย่างรวดเร็วไม่ต้องใช้การประมวลผลใดๆ อีก (เนื่องจากไฟล์เท็กซ์เจอร์จะมีขนาดไม่ใหญ่อยู่แล้วโดยปกติจะมีขนาด 256*256 พิกเซล ขนาด 32 บิต หรือ 256 kBytes)

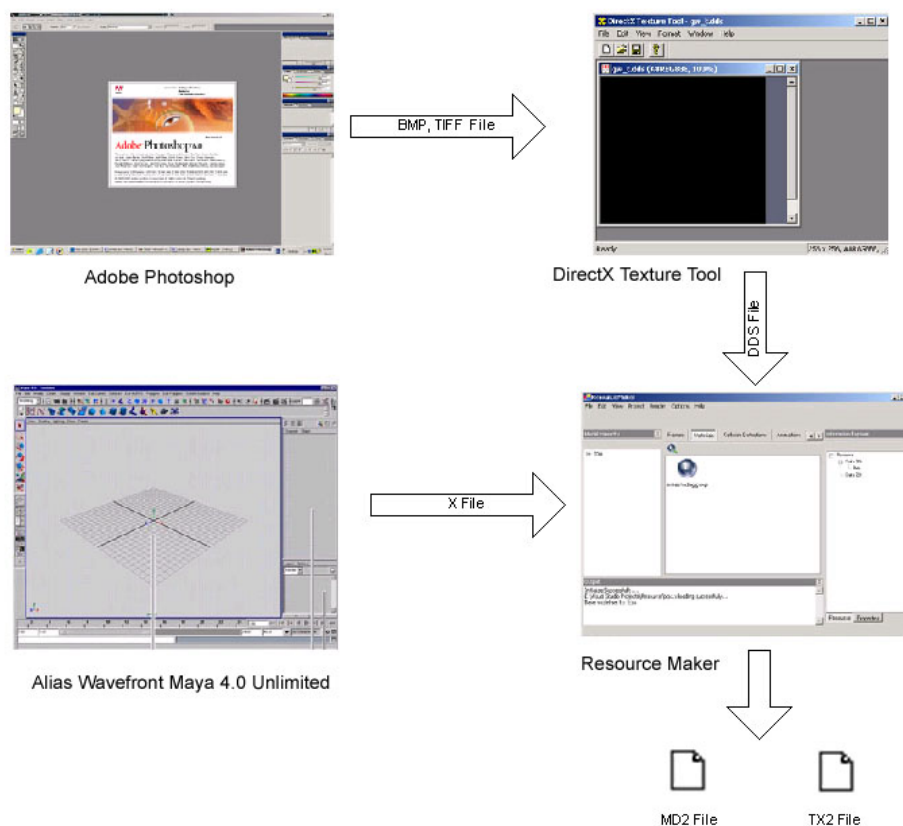
ซึ่งจากการค้นหารูปแบบไฟล์ต่างๆ ในปัจจุบันพบว่า ไฟล์โมเดล 3 มิติ ส่วนมากจะไม่มีข้อมูลเอนิเมชัน และบางส่วนของข้อมูลเอนิเมชัน แต่มักจะเป็นเอนิเมชันในแบบเวอร์เท็กซ์เบลนดิง ซึ่งไม่ใช่ในโครงการนี้ หลังจากตรวจสอบคุณสมบัติของรูปแบบไฟล์ชนิดต่างๆ แล้วพบว่า ไฟล์ในรูปแบบของ .X มีคุณสมบัติใกล้เคียงกับรูปแบบของไฟล์ที่ต้องการมากที่สุด แต่มีบางส่วนซึ่งไม่ตรงกันความต้องการ ถึงแม้ว่า SDK ของไคลร์เอ็กซ์จะมีไลบรารีในส่วนที่จัดการอ่านและเขียนข้อมูลอยู่แล้วซึ่งทำให้่ง่ายในการใช้งาน แต่การเข้าถึงข้อมูลต่างๆ ที่อยู่ในเวอร์เท็กซ์บัฟเฟอร์จะถูกจำกัด และลักษณะการเก็บข้อมูลเอนิเมชันไม่ได้เก็บข้อมูลแต่ละท่วงท่าแยกจากกันทำให้มีปัญหาในการสร้างโมเดล 3 มิติ

ดังนั้น จากความต้องการต่างๆ ข้างต้น โครงการนี้จึงแบ่งออกเป็นองค์ประกอบย่อยๆ 3 ส่วนดังนี้

1. การพัฒนาโปรแกรมสำหรับอ่านข้อมูลจากไฟล์ในรูปแบบต่างๆ ที่ได้จากแอปพลิเคชันสร้างโมเดล 3 มิติ และเขียนข้อมูลลงในแบบที่ต้องการ
2. การพัฒนาเกมเอนจินซึ่งจะให้ เป็นไลบรารีในการเขียนแอปพลิเคชันเกม
3. ใช้ไลบรารีจากเกมเอนจินในการพัฒนาแอปพลิเคชันเกม

สรุปขั้นตอนในการสร้างไฟล์โมเดลและเทกเจอร์ที่จะใช้ในการพัฒนาแอปพลิเคชันเกมในโครงการนี้

เริ่มต้นจากการขึ้นโมเดลในแอปพลิเคชันสร้างโมเดล 3 มิติ เช่น Maya, 3D Studio Max เป็นต้น แล้วส่งข้อมูลออกมาในรูปของ X ไฟล์ ส่วนภาพที่จะใช้เป็นเทกเจอร์สร้างขึ้นโดยใช้โปรแกรม Adobe® Photoshop, ดาวน์โหลดจากอินเทอร์เน็ตหรือสแกนเข้ามาจากเครื่องสแกน ซึ่งภาพที่ได้มาส่วนใหญ่มักจะไม่ตรงกับความต้องการจึงต้องมาแต่งภาพใน Adobe® PhotoShop ก่อน หลังจากนั้นนำข้อมูลในรูปของไฟล์ต่างๆ ที่ได้มาแปลงให้อยู่ในรูปแบบที่จะใช้งานเก็บเอาไว้ ส่วนของแอปพลิเคชันเขียนขึ้นโดยใช้ไลบรารีในเกมเอนจินที่เขียนขึ้นมา ดังแผนภาพ



รูปที่ 11-2 ขั้นตอนการสร้างรีซอร์สที่ใช้ในการพัฒนาเกม

11.2 การออกแบบ

11.2.1 การพัฒนาแอปพลิเคชันที่จัดการกับข้อมูลจากแอปพลิเคชันสร้างโมเดล 3 มิติ

ในอุตสาหกรรมการพัฒนาซอฟต์แวร์เกม ขั้นตอนนี้คือการพัฒนาทูล (Tool) ซึ่งจะช่วยให้การพัฒนาซอฟต์แวร์เกมเป็นไปได้อย่างรวดเร็วและมีประสิทธิภาพ ซึ่งการพัฒนาทูลไม่ได้จำกัดว่าจะต้องเป็นโปรแกรมที่มีประสิทธิภาพมากมายนัก จึงควรเลือกพัฒนาด้วยภาษาโปรแกรมที่ง่ายและสามารถเขียนแอปพลิเคชันแบบ GUI ได้เป็นอย่างดี เช่น Visual Basic, Java เป็นต้น ซึ่งในโครงการนี้ได้เลือกใช้ Visual C# ซึ่งแม้จะอยู่ในขั้นเบต้าแต่ก็สามารถตอบสนองความต้องการของผู้พัฒนาโครงการได้

เหตุผลที่ใช้ Microsoft® Visual® C# ในการพัฒนามีดังนี้

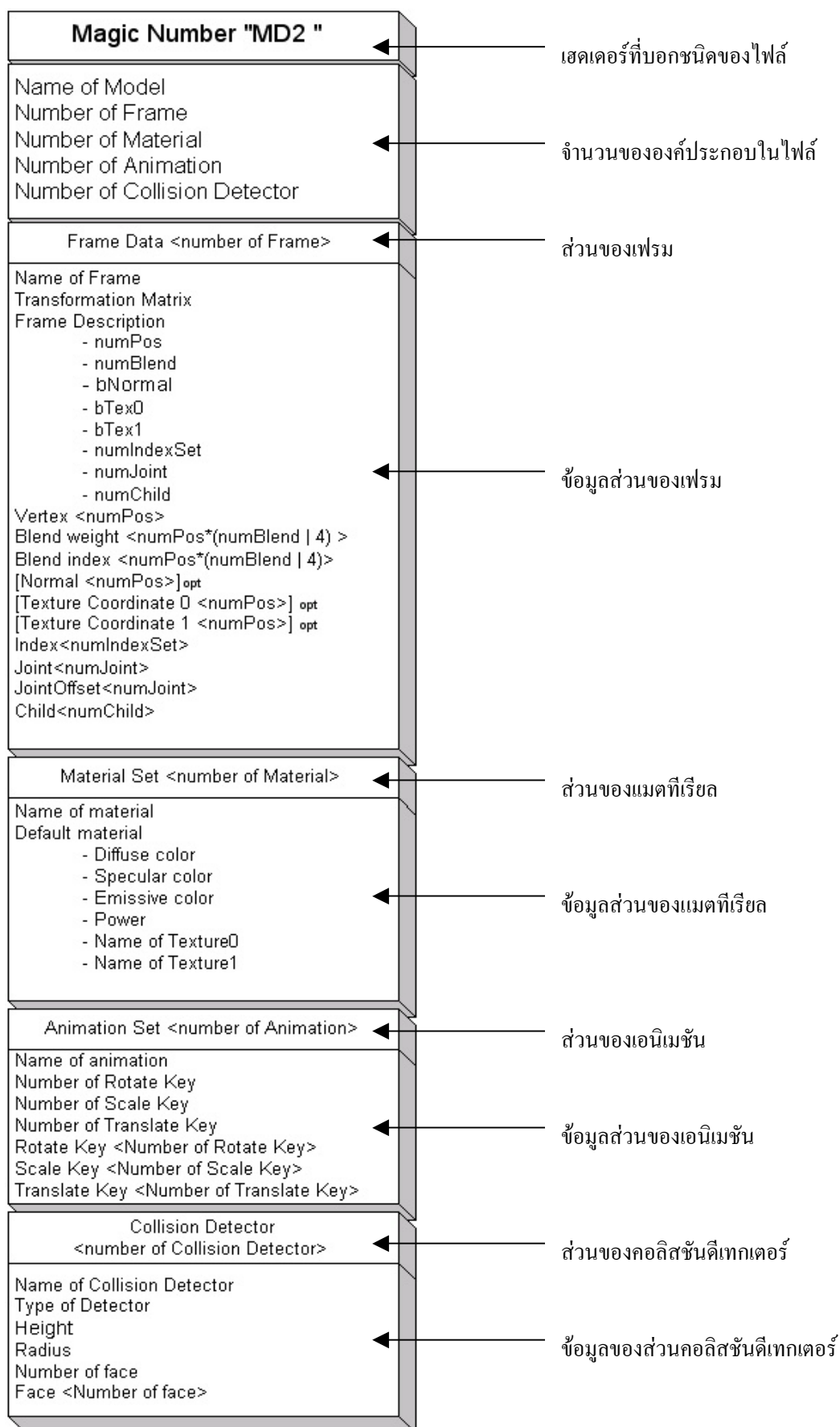
1. เป็นภาษาที่ทำงานด้าน GUI ได้เป็นอย่างดี เนื่องจากมีเฟรมเวิร์กที่สนับสนุนงานด้าน GUI ที่มีประสิทธิภาพ
2. เป็นภาษาที่มีสิ่งแวดล้อมในการพัฒนา (IDE) คล้ายคลึงกับการใช้งาน Visual C++ ซึ่งใช้ในการพัฒนาเป็นหลักในโครงการนี้
3. มีโครงสร้างภาษาเป็น OOP คล้ายคลึงกับภาษาจาวา และภาษา C++ ทำให้ใช้เวลาในการเรียนรู้ น้อย สามารถใช้งานได้ทันที

หน้าที่ของแอปพลิเคชันแปลงข้อมูลจากไฟล์ในรูปแบบต่างๆ

1. อ่านข้อมูลจากไฟล์ในรูปแบบของไฟล์นามสกุล X แล้วนำข้อมูลเวอร์เท็กซ์มาจัดเรียงให้อยู่ในรูปแบบที่สามารถใช้งานได้ทันทีโดยไดเรกทรีดี ดังรูปที่
2. เพิ่มข้อมูลในส่วนของตำแหน่งของเทกเจอร์อีก 1 ชั้น โดยข้อมูลจะมาจากอีกโมเดลหนึ่งในกรณีที่ต้องการใช้งานมัลติเทกเจอร์
3. เพิ่มข้อมูลในส่วนของอนิเมชันโดยข้อมูลอนิเมชันจะนำมาจากอีกโมเดลหนึ่ง
4. ใส่ชื่อของเทกเจอร์ที่ต้องการให้กับแมตทีเรียล (ไม่ใช่ชื่อของเทกเจอร์ที่มากับไฟล์เนื่องจากไฟล์เทกเจอร์ที่ใช้ในแอปพลิเคชัน 3 มิติมีรูปแบบไม่ตรงกับที่จะใช้ในเกม)
5. เขียนข้อมูลจากไฟล์ในรูปแบบ MD2 ซึ่งกำหนดขึ้นมา

ขั้นตอนของข้อมูลจากไฟล์รูปแบบ X ที่ต้องการการประมวลผล

1. นำข้อมูลมาประกอบเป็นเวอร์เท็กซ์
2. ปรับค่าอินเด็กซ์ที่มากกว่า 3 ค่า ให้เป็น 3 ค่า และลบค่าอินเด็กซ์ที่มีน้อยกว่า 3 ค่าทิ้งไป
3. นำแมตทีเรียลมาเป็นคุณสมบัติในระดับโมเดล (เนื่องจากใน X File แมตทีเรียลเป็นคุณสมบัติระดับเฟรม)
4. นำข้อมูลที่อยู่ในส่วนของ SkinWeights มาจัดเรียงจากมากไปน้อย เก็บชื่อของ Joint ที่เกี่ยวข้องไว้เพื่อหาค่าอินเด็กซ์ในการอ้างอิงจากระดับโมเดล



รูปที่ 11-3 ลำดับการเก็บข้อมูลในไฟล์รูปแบบ MD2

11.2.2 การพัฒนาเกมเอนจิน

ความต้องการ (Requirement)

แอปพลิเคชันที่ใช้งานเอนจินจะต้องใช้งานฟังก์ชันและออปเจกต์ได้อย่างง่าย ๆ

คำว่าอย่างง่าย ๆ ในโครงงานนี้มีหมายถึง สามารถมองข้อมูลของโมเดลเป็นวัตถุที่สามารถแสดงผลได้ทันทีโดยแอปพลิเคชันไม่ต้องคำนึงถึงวิธีการที่จะทำให้มันแสดงผลเลย

การออกแบบ (Design)

เกมเอนจิน จะสร้างขึ้นโดยแบ่งเป็นโมดูลที่ทำหน้าที่ในด้านที่แตกต่างกันเฉพาะในส่วนที่ต้องการใช้ เช่น งานด้านแสดงผลกราฟิก, งานด้านการควบคุมเสียง, งานด้านการควบคุมการเล่นไฟล์วีดีโอ และงานด้านการจัดการอุปกรณ์อินพุต โดยคลาสที่ประกอบอยู่ในแต่ละโมดูลจะสร้างขึ้นโดยการรวบรวมอินเทอร์เฟซและเมทอดที่ทำงานพื้นฐานในไลบรารีเอ็กซ์ให้เป็นคลาสที่ทำงานอย่างใดอย่างหนึ่งและอยู่ในรูปที่สามารถใช้งานได้ง่ายที่สุด

ในโครงงานนี้ได้แบ่งส่วนของเอนจินออกเป็นโมดูล ดังนี้

11.2.2.1 ส่วนของการแสดงผลกราฟิก

ควบคุมการแสดงผลทั้งในแบบของ 2 มิติและ 3 มิติ โดยทำงานติดต่อกับการ์ดแสดงผลผ่านทางไลบรารีดี แต่เนื่องจากไลบรารีดีเป็น API ขั้นพื้นฐาน ซึ่งประกอบขึ้นด้วยอินเทอร์เฟซและเมทอดที่ทำงานในระดับองค์ประกอบย่อย จึงมีความจำเป็นที่จะต้องรวบรวมและจัดการกับอินเทอร์เฟซและเมทอดเหล่านี้ให้ใช้งานได้ง่ายและมีความผิดพลาดน้อยโดยการนำหลักการของ OOP เข้ามาใช้

เอนจินในส่วนของการแสดงผลจะเป็นการนำเอาอินเทอร์เฟซและเมทอดที่มีอยู่ในไลบรารีดีมาสร้างเป็นคลาส เพื่อที่จะใช้ในการทำงานเฉพาะอย่าง เช่น การโหลดโมเดล, การเตรียมบัฟเฟอร์สำหรับข้อมูลที่จะใช้ในการเรนเดอร์ เป็นต้น

คุณสมบัติที่ต้องการ (Requirement)

- ด้านการทำงาน
 1. มีความสามารถในการทำภาพเอนิเมชันโมเดล 3 มิติ
 2. มีความสามารถในการตรวจสอบการชนของวัตถุ
 3. มีความสามารถในการแสดงผลภาพ 2 มิติและการทำเอนิเมชันภาพ 2 มิติและเทกเจอร์ (Sprite)
 4. มีประสิทธิภาพในการทำงานที่ดี (ไม่ต่ำกว่า 30 FPS ในสภาพแวดล้อมที่กำหนด)
- ด้านการใช้งาน

สามารถใช้งานฟังก์ชันและคลาสในการทำงานต่างๆ ได้ง่ายค้าย เช่น ต้องการแสดงผลโมเดล 3 มิติ 1 ชิ้น มีขั้นตอนการทำงาน คือ โหลดข้อมูลจากไฟล์และส่งให้แสดงผลเท่านั้น

การออกแบบ

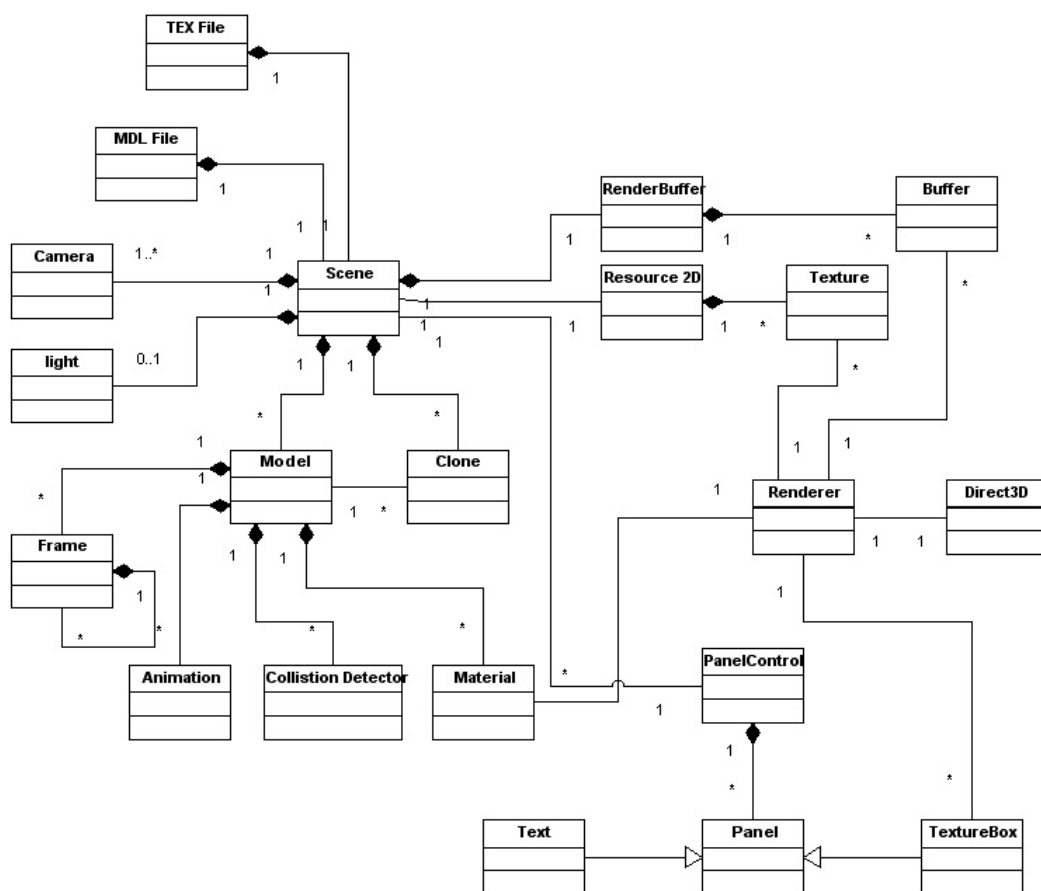
1. กำหนดวัตถุ

เนื่องจากแอปพลิเคชันของเราทำงานกับข้อมูลทั้งหมดไม่มีส่วนที่ติดต่อกับฮาร์ดแวร์โดยตรง (ได้เรียกสามดีทำงานในส่วนที่ติดต่อกับฮาร์ดแวร์ให้แล้ว) การกำหนดวัตถุจึงใช้ข้อมูลที่เก็บเป็นตัวกำหนด

วัตถุ	ข้อมูล	ระยะเวลาในการเก็บ
เฟรม (Frame)	เก็บข้อมูลเวอร์เท็กซ์ซึ่งใช้ในการกำหนดตำแหน่งของจุด (position), ค่าของเวกเตอร์ตั้งฉาก, การเรียงตัวของจุด, ค่าตำแหน่งของเทกเจอร์ เป็นต้น	ช่วงเวลาที่มีการแสดงโมเดลนั้นในเกม
แอนิเมชัน (Animation)	เก็บข้อมูลสำหรับการกำหนดค่าการเคลื่อนไหว, การหมุน, การย่อขยาย และค่าเวลา ที่จะใช้ในการคำนวณการเคลื่อนไหวของโมเดล	ช่วงเวลาที่โมเดลนั้นในเกม
คอลลิชันดีเทกเตอร์ (Collision Detector)	เก็บข้อมูลสำหรับการตรวจสอบการชนของวัตถุ เช่น ขอบเขต, ชนิดของการตรวจสอบ เป็นต้น	ช่วงเวลาที่โมเดลนั้นในเกม
แมตทีเรียล (Material)	เก็บข้อมูลของแมตทีเรียลที่ใช้ในการกำหนดลักษณะการคำนวณทางแสงของโมเดล	ช่วงเวลาที่โมเดลนั้นในเกม
เทกเจอร์ (Texture)	เก็บข้อมูลของภาพ 2 มิติและข้อมูลที่จะใช้แสดงแอนิเมชันของภาพ 2 มิติ	ช่วงเวลาที่ต้องการแสดงภาพ 2 มิติ
ไดเรกทรีดี (Direct3D)	เก็บสถานะและคุณสมบัติต่างๆ ที่ได้กำหนดให้กับไดเรกทรีดี	ช่วงที่แอปพลิเคชันทำงาน
โมเดล (Model)	เก็บข้อมูลที่แสดงความสัมพันธ์ระหว่างแต่ละเฟรม และความสัมพันธ์ระหว่างแอนิเมชัน, แมตทีเรียล และคอลลิชันดีเทกเตอร์	ช่วงเวลาที่มีการแสดงโมเดลนั้นในเกม
โคลน (Clone)	เก็บข้อมูลที่อ้างอิงไปยังข้อมูลของโมเดล และข้อมูลสำหรับแต่ละโมเดลที่จะถูกแสดง	เมื่อต้องการแสดงโมเดลแต่ละโมเดล
พานอลคอนโทรล (PanelControl)	เก็บข้อมูลและควบคุมการแสดงผลภาพ 2 มิติ	ช่วงเวลาที่แอปพลิเคชันทำงาน
พานอล (Panel)	เก็บข้อมูลที่แสดงขอบเขตพื้นที่ในการแสดงผลภาพ 2 มิติ	ช่วงเวลาที่ต้องการแสดงผลภาพ 2 มิติ
บัฟเฟอร์ (Buffer)	เก็บข้อมูลของเวอร์เท็กซ์ที่สามารถส่งไปเรนเดอร์ได้โดยไดเรกทรีดี	ช่วงเวลาที่ต้องการแสดงโมเดลนั้นในเกม
เรนเดอร์เรอร์ (Renderer)	เก็บข้อมูลและจัดการให้มีการเรนเดอร์อย่างถูกต้องและมีประสิทธิภาพ	ช่วงเวลาที่แอปพลิเคชันทำงาน

เรนเดอร์บัฟเฟอร์ (RenderBuffer)	เก็บข้อมูลที่แสดงสถานะและควบคุมการทำงานของบัฟเฟอร์	ช่วงเวลาที่มีการใช้งานบัฟเฟอร์
กล้อง (Camera)	เก็บข้อมูลสถานะของกล้อง เช่น ตำแหน่ง, โพกัส เป็นต้น	ช่วงเวลาที่ต้องการใช้กล้อง
แสง (Light)	เก็บข้อมูลสถานะของแสง	ช่วงเวลาที่ต้องการใช้แสง
ฉาก (Scene)	เก็บข้อมูลความสัมพันธ์ของวัตถุต่างๆ ที่จะปรากฏขึ้นในการแสดงผลหนึ่งๆ เช่น โมเดล(โคลน), กล้อง, แสง และ ภาพ 2 มิติ เป็นต้น	ช่วงเวลาที่ต้องการแสดงผลภาพ
ไฟล์โมเดล (Model File)	เก็บข้อมูลและฟังก์ชันที่ใช้ในการโหลดไฟล์โมเดล	ช่วงเวลาที่ต้องการโหลดไฟล์โมเดล
ไฟล์เทกเจอร์ (Texture File)	เก็บข้อมูลและฟังก์ชันที่ใช้ในการโหลดไฟล์เทกเจอร์	ช่วงเวลาที่ต้องการโหลดไฟล์เทกเจอร์

ตารางที่ 11-1 วัตถุที่ได้จากการแยกแยะโดยใช้ข้อมูลที่เก็บสำหรับส่วนแสดงผล



รูปที่ 11-4 คลาสไดอะแกรมในส่วนของการกราฟิก

รายละเอียดในคลาสที่มีความสำคัญ

Direct3D (class NG3D ใน ng3d.h)

ทำหน้าที่ในส่วนที่เกี่ยวข้องทั้งหมด เช่น การระบุอุปกรณ์ที่มีอยู่ในเครื่องคอมพิวเตอร์, การตรวจสอบความสามารถของอุปกรณ์ และการสร้างอุปกรณ์สำหรับการแสดงผล ซึ่งคลาสนี้ผู้พัฒนาได้ใช้เทคนิคเพื่อให้สามารถสร้างคลาสนี้ได้เพื่อหนึ่งคลาสในหนึ่งแอปพลิเคชันเท่านั้น

โครงสร้างข้อมูล NGCREATEINFO เป็นข้อมูลที่ส่งให้กับคลาส NG3D เพื่อสร้างอุปกรณ์ในการแสดงผลที่ตามที่กำหนดซึ่งถ้าไม่สามารถหาอุปกรณ์ที่กำหนดได้ คลาสจะใช้ค่าโดยปริยายในการสร้างอุปกรณ์

โครงสร้างข้อมูล NGCREATEINFO

```
typedef struct NGCREATEINFO
{
    HWND      hWnd;

    DWORD     dwAdapter;
    DWORD     dwWidth;
    DWORD     dwHeight;
    DWORD     dwMode;

    DWORD     dwMinDepthBits;
    DWORD     dwMinStencilBits;
    DWORD     dwNumDepthBuffer;

    BOOL      bWindow;
    BOOL      bCursor;
    BOOL      bUseDepthBuffer;
} NGCREATEINFO, *LPNGCREATEINFO;
```

ตารางที่ 11-2 ข้อมูลในโครงสร้างข้อมูล NGCREATEINFO

สมาชิก

hWnd	ค่าแอสเคลของวินโดว์ที่ต้องการจะใช้เรนเดอร์
dwAdapter	ค่าแสดงหมายเลขของอุปกรณ์ที่จะใช้ซึ่งคุณสมบัติของอุปกรณ์จะได้มาจากการขั้นตอนการระบุอุปกรณ์ ในกรณีที่ต้องการใช้อุปกรณ์โดยปริยายให้ใส่ค่าเป็น 0
dwWidth	ค่าความกว้างของพื้นที่แสดงผลซึ่งจะมีผลเมื่อแสดงผลในแบบเต็มจอเท่านั้น
dwHeight	ค่าความสูงของพื้นที่แสดงผลซึ่งจะมีผลเมื่อแสดงผลในแบบเต็มจอเท่านั้น
dwMode	ค่าแสดงคุณลักษณะของสีที่จะแสดงผล โดยปกติจะมีค่าเป็น 16 ในกรณีที่ต้องการแสดงผลสีแบบ 16 บิต หรือ 32 ในกรณีที่ต้องการแสดงผลแบบ 32 บิต
dwMinDepthBits	ค่าแสดงบิตของเดฟบัฟเฟอร์ขั้นต่ำที่สามารถใช้งานได้โดยปกติกำหนดค่าไว้ที่ 16 หรือ 16 บิต
dwMinStencilBits	ค่าแสดงบิตของสเตนซิลบัฟเฟอร์ขั้นต่ำที่สามารถใช้งานได้โดยปกติมักจะไม่ใช่ใช้งานจึงกำหนดค่าไว้ที่ 0
dwNumDepthBuffer	ค่าแสดงจำนวนของเดฟบัฟเฟอร์ปกติกำหนดค่าไว้ที่ 1 สามารถกำหนดค่ามากกว่านี้ได้ซึ่งจะได้ภาพที่มีความนุ่มนวลขึ้นแต่จะใช้หน่วยความจำจำนวนมาก
bWindow	ให้แสดงผลในแบบวินโดว์หรือไม่
bCursor	ให้แสดงฮาร์ดแวร์เคอร์เซอร์เมื่อแสดงผลแบบเต็มจอหรือไม่

bUseDepthBuffer ให้มีการใช้งานเคฟบัฟเฟอร์หรือไม่

โครงสร้างของคลาส NG3D

```
class NG3D
{
protected:
    NGCREATEINFO          m_CreateInfo;
    RECT                  m_rcClient;
    DWORD                 m_dwNumAdapters;
    DWORD                 m_dwAdapter;
    DWORD                 m_dwBehavior;

    LPDIRECT3D8           m_lpD3d;
    LPDIRECT3DDEVICE8     m_lpDevice;

    D3DPRESENT_PARAMETERS m_d3dpp;
    D3DCAPS8              m_d3dCaps;

    LPDIRECT3DSURFACE8    m_lpBackBuffer;
    D3DSURFACE_DESC       m_d3dsdBackBuffer;

    NGADAPTERINFO          m_AdaptersInfo[8];
    NGMODEINFO*            m_pModeInfo;

    NG3D(NGCREATEINFO*);

    BOOL                  FindDepthStencilFormat(UINT,
        D3DDEVTYPE, D3DFORMAT, D3DFORMAT*);
    HRESULT                InitialGraphics();
    HRESULT                InitialDevice();
    HRESULT                CheckDevice();
    HRESULT                FindPreferDevice();

    static NG3D*           m_cGraphics;

public:
    ~NG3D();
    static HRESULT          CreateGraphics(NGCREATEINFO*);
    HRESULT                ResizeScene(RECT*);
    BOOL                  TestAdapter(NGADAPTERINFO*&, INT&);

    virtual HRESULT          ConfirmDevice(D3DCAPS8*, DWORD, D3DFORMAT);
    virtual HRESULT          ClearResource();
    virtual HRESULT          RestoreResource();

    // Accessors
    LPDIRECT3DDEVICE8       GetDevice;
    NGADAPTERINFO*          GetAdapterInfo(UINT id);
    DWORD                   GetAdapterCount();
    D3DCAPS8*               GetDeviceCaps();
    NGCREATEINFO*           GetCreateInfo();

    static NG3D*            GetGraphics();
};

inline NG3D* Graphics()
{
    return NG3D::GetGraphics();
}
```

ตารางที่ 11-3 โครงสร้างในคลาส NG3D

ฟังก์ชันที่สำคัญ

static HRESULT CreateGraphics(NGCREATEINFO* createInfo)

ทำการสร้างอุปกรณ์สำหรับการแสดงผลตามค่าที่กำหนดในโครงสร้างข้อมูล NGCREATEINFO
พารามิเตอร์

createInfo ค่าพอยเตอร์ที่ชี้ไปยังโครงสร้างข้อมูล NGCREATEINFO ซึ่งกำหนดลักษณะของ
อุปกรณ์แสดงผลที่จะสร้าง

หมายเหตุ คลาส NG3D มีวิธีการใช้งานต่างจากคลาสอื่น เนื่องจากคลาสนี้เป็นคลาสแบบสแตติก การใช้งาน
จึงต้องทำตามขั้นตอนดังนี้

```
#include "ng3d.h"
.
.
.
static NG3D*   g_NG3d = NULL;
.
.
.

// Create Direct3D (NG3D)
NGCREATEINFO createInfo;
NG3D::CreateGraphics(createInfo);
g_NG3d = Graphics()->GetGraphics();// Graphics() is a pointer to NG3D object
```

ตารางที่ 11-4 การสร้างวัตถุของคลาส NG3D

นอกจากนั้นแล้ว ฟังก์ชัน Graphics() สามารถนำไปใช้ได้ก็ได้ที่ include “ng3d.h” ไว้ ซึ่งการใช้
ฟังก์ชันนี้เมื่อกับการใช้งานพอยเตอร์ที่ชี้มายังออบเจกต์ NG3D

BOOL TestAdapter(NGADAPTERINFO*& adapterInfo, INT& numAdapter)

เป็นฟังก์ชันที่ใช้ในการทดสอบอุปกรณ์แสดงผลที่มีอยู่ในเครื่องคอมพิวเตอร์
พารามิเตอร์

adapterInfo ค่าพอยเตอร์ที่ชี้ไปยังโครงสร้างข้อมูล NGADAPTERINFO ซึ่งจะรับข้อมูลออกไป

numAdapter จำนวนของอุปกรณ์ที่ตรวจสอบพบ

ส่งค่ากลับ

ฟังก์ชันจะส่งค่า TRUE ถ้ามีการพบอุปกรณ์อย่างน้อย 1 ตัว และ FALSE ถ้าไม่พบอุปกรณ์ใดๆ
โครงสร้างข้อมูล NGADAPTERINFO

โครงสร้างข้อมูลนี้ใช้ในการเก็บข้อมูลของอุปกรณ์แสดงผลแต่ละตัวที่พบ

```
typedef struct NGADAPTERINFO
{
    D3DADAPTER_IDENTIFIER8    d3dAdapterID;
    D3DDISPLAYMODE            d3ddmDesktop;
    D3DMULTISAMPLE_TYPE        d3dMultiSampleType;
    D3DMULTISAMPLE_TYPE        d3dWindowMultiSampleType;
    NGDEVICEINFO               devices;
    DWORD                      dwNumModes;
    NGMODEINFO                 modes[100];
} NGADAPTERINFO, *LPNGADAPTERINFO;
```

ตารางที่ 11-5 ข้อมูลที่เก็บในโครงสร้างข้อมูล NGADAPTERINFO

สมาชิก

d3dAdapterID	เก็บข้อมูลที่อธิบายที่มาของการ์ดแสดงผล เช่น ชื่ออุปกรณ์, รุ่น, ผู้ผลิต, ผู้จำหน่าย และไดรเวอร์ เป็นต้น
d3ddmDesktop	รีโซลูชันของการแสดงผลในปัจจุบัน
d3dMultiSampleType	ความสามารถในการทำ Anti-Aliasing ในการแสดงผลแบบเต็มจอ
d3dWindowMultiSampleType	ความสามารถในการทำ Anti-Aliasing ในแสดงผลแบบวินโดว์
devices	รายละเอียดของอุปกรณ์ที่มีอยู่บนการ์ด (มีให้เลือกเฉพาะในแบบฮาร์ดแวร์)
dwNumModes	จำนวนของรีโซลูชันที่สามารถแสดงผลได้
modes	เก็บข้อมูลรีโซลูชันที่สามารถแสดงผลได้

Renderer (คลาส NGRenderer ใน “ngrender.h”)

คลาส NGRenderer ทำหน้าที่ในการจัดระเบียบการเรนเดอร์ให้เป็นไปอย่างถูกต้อง โดยคลาส NGClone หรือคลาส NGTextureBox ที่ต้องการจะแสดงผลจะต้องส่งข้อมูลมาให้กับ Renderer ทุกครั้งที่ต้องการจะแสดงผล หรือส่งข้อมูลมาเก็บอย่างถาวรก็ได้ในกรณีที่ต้องการแสดงผลตลอดเวลา โดย Renderer จะเป็นผู้ไปขอสิ่งที่จำเป็นในการเรนเดอร์ภาพจะออกไปอีกัดอื่นๆ เช่น ขอดำแหน่งจาก Frame, ขอค่าของแมตทริกซ์, ขอค่าพอยเตอร์ของเทกเจอร์ เป็นต้น แล้วส่งให้บัฟเฟอร์ที่เก็บข้อมูลเรนเดอร์ภาพโดยส่งค่าที่จำเป็นไปให้

โครงสร้างของคลาส NGRenderer

```
class NGRenderer
{
private:
    NGRENDERDESC*      m_rdesc[1024];
    NGRENDERDESC*      m_tbdesc[1024];
    INT                 m_index;
    INT                 m_index2;
    NG3D*               m_p3d;
    LPDIRECT3DDEVICE8   m_pDevice;
    NGBuffer*           m_curBuffer;
    INT                 m_curTex0;
    INT                 m_curTex1;
    DWORD               m_dwFVF;
    NGRes2d*            m_res2d;
    INT                 m_Size2d;
    NGRenderer*         Initial(NG3D*);
    static NGRenderer*  m_this;

public:
    NGRenderer();

    BOOL                Clear();
    BOOL                Add(NGRENDERDESC*);
    BOOL                Sort();
    BOOL                Render();
    INT                 Length();
    NGRENDERDESC**      GetRenderDesc();
    VOID                SetRes2d(NGRes2d*);

    static NGRenderer*  CreateRenderer(NG3D*);
    static NGRenderer*  GetRenderer();
};

inline NGRenderer* Render()
{
    return NGRenderer::GetRenderer();
};
```

ตารางที่ 11-6 โครงสร้างคลาส NGRenderer

ฟังก์ชันที่สำคัญ

static NGRenderer* CreateRenderer(NG3D* p3d)

เป็นฟังก์ชันที่ใช้ในการสร้างอินสแตนซ์ของคลาสนี้

หมายเหตุ เช่นเดียวกับคลาส NG3D ออปเจ็กต์ของคลาสนี้จะมีได้เพียง 1 ออปเจ็กต์ใน 1 แอปพลิเคชัน เนื่องจากเป็นตัวแทนที่จะจัดการให้มีการเรนเดอร์ภาพอย่างมีประสิทธิภาพและมีความถูกต้อง คลาสนี้มีวิธีการสร้างแบบเดียวกับคลาส NG3D แต่ใช้ Render() ในการใช้งานแทน

พารามิเตอร์

p3d อินสแตนซ์ของคลาส NG3D สร้างใส่ฟังก์ชัน Graphics() ในพารามิเตอร์นี้ได้

BOOL Add(NGRENDERDESC* renderDesc)

ฟังก์ชันนี้ทำหน้าที่เพิ่มงานในการเรนเดอร์โดยข้อมูลในโครงสร้างข้อมูล NGRENDERDESC จะใช้ในการเรียงลำดับ, นำไปใช้ซอร์ซอร์สจากวัตถุอื่นๆ และใช้เป็นพารามิเตอร์ในการเรนเดอร์ภาพโดยออปเจ็กต์ของคลาส NGBuffer ที่เก็บข้อมูลเวกเตอร์ที่ใช้ในการเรนเดอร์ภาพ

พารามิเตอร์

renderDesc พอยเตอร์ที่ชี้ไปยังโครงสร้างข้อมูล NGRENDERDESC ซึ่งใช้ในการกำหนดการเรนเดอร์

โครงสร้างข้อมูล NGRENDERDESC

```
struct NGRENDERDESC
{
    NGBuffer*          pBuffer;
    INT                vStart;
    INT                vCount;
    INT                iBuffer;
    INT                iStart;
    INT                iCount;
    INT                iMtrl;
    INT                t0;
    INT                t1;
    INT                blend;
    D3DMATERIAL8*      mtrl;
    D3DMATRIX*         mat;
    ArrayList<NGFRAMEX*> joint;
    ArrayList<D3DXMATRIX*> jOffset;
};
```

ตารางที่ 11-7 ข้อมูลที่เก็บรักษาในโครงสร้างข้อมูล NGRENDERDESC

สมาชิก

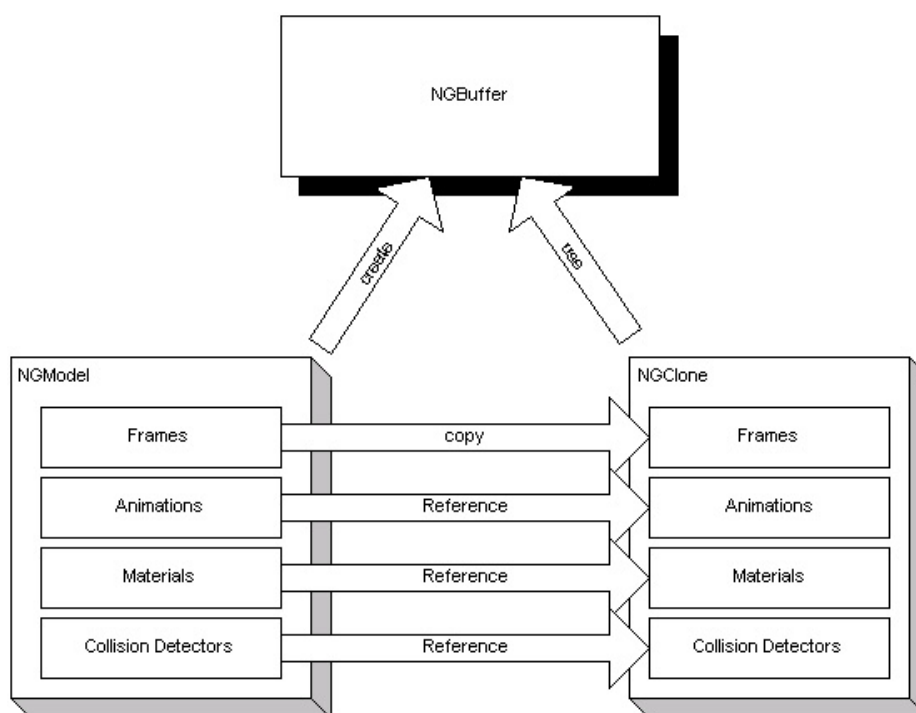
pBuffer	พอยเตอร์ที่ชี้ไปยังวัตถุของ NGBuffer ซึ่งเก็บข้อมูลของเวอร์เท็กซ์และอินเด็กซ์ที่จะเรนเดอร์บนพื้นที่แสดงผล
vStart	ตำแหน่งเริ่มต้นที่จะเรนเดอร์ในเวอร์เท็กซ์บัฟเฟอร์
vCount	จำนวนของเวอร์เท็กซ์ในเวอร์เท็กซ์บัฟเฟอร์
iBuffer	หมายเลขของอินเด็กซ์บัฟเฟอร์ที่ใช้ในการเรนเดอร์ข้อมูลชุดนี้
iStart	ตำแหน่งเริ่มต้นของข้อมูลในอินเด็กซ์บัฟเฟอร์
iCount	จำนวนโพรมิทีฟที่จะเรนเดอร์
iMtrl	หมายเลขของแมตทีเรียลใช้อ้างอิงในการขอข้อมูลแมตทีเรียล
t0	หมายเลขของเทกเจอร์ใช้อ้างอิงในการขอข้อมูลเทกเจอร์
t1	หมายเลขของเทกเจอร์ใช้อ้างอิงในการขอข้อมูลเทกเจอร์
blend	จำนวนของข้อมูลที่ใช้ในการทำอินเด็กซ์เวอร์เท็กซ์เบลนดิง
mtrl	พอยเตอร์ที่ชี้ไปยังข้อมูลแมตทีเรียล
mat	พอยเตอร์ที่ชี้ไปยังข้อมูลตำแหน่ง
joint	พอยเตอร์ที่ชี้ไปยังตำแหน่งของโบน
jointOffset	พอยเตอร์ที่ชี้ไปยังตำแหน่งที่เก็บค่าออฟเซตของโบน

Model และ Clone (คลาส NGModel และ NGClone ใน “ngmodel.h”)

โดยปกติแล้วในแอปพลิเคชันเกมจำนวนมาก โมเดลที่ปรากฏในฉากหนึ่งๆ มักมีข้อมูลของเวอร์เท็กซ์และอินเด็กซ์ที่ใช้ในการกำหนดรูปร่างของโมเดลที่เหมือนกัน ทั้งนี้เนื่องมาจากการออกแบบโมเดลให้หลากหลายเป็นงานที่ต้องใช้จินตนาการอย่างมาก, ใช้เวลานาน และเสียค่าใช้จ่ายมาก (ค่าจ้างนักออกแบบกราฟิก 3 มิติที่มีฝีมือมีราคาแพงมาก) นอกจากนั้น การมีโมเดลจำนวนมากยังสิ้นเปลืองหน่วยความจำและทำให้แอปพลิเคชันทำงานได้ไม่มีประสิทธิภาพเนื่องจากต้องติดต่อกับดิสต์ในการอ่านไฟล์จำนวนมาก ดังนั้น ในโครงการนี้จึงคิดวิธีการในการใช้งานโมเดลหนึ่งให้สามารถแสดงผลออกมาเสมือนเป็นหลายโมเดลโดยสร้างคลาส NGClone ขึ้นมา

คลาส NGClone จะทำการคัดลอกข้อมูลที่ทำให้แต่ละโมเดลมีความแตกต่างกันออกมาจากคลาส NGModel เช่น ข้อมูลที่เกี่ยวข้องกับตำแหน่งของโมเดล ส่วนข้อมูลอื่นที่ไม่ทำให้เกิดความแตกต่างจะใช้งานร่วมกัน เช่น ข้อมูลของแมตทีเรียล, ข้อมูลของแอนิเมชัน และข้อมูลของคอลลิชันดีเทกเตอร์ เป็นต้น นั่นคือ คลาส NGModel ในโครงการนี้จะทำหน้าที่เป็นต้นแบบในการสร้างคลาส NGClone ซึ่งจะเป็นโมเดลที่ใช้ในการแสดงผลจริง

ความสัมพันธ์ระหว่างคลาส NGModel และคลาส NGClone



รูปที่ 11- 5 แสดงความสัมพันธ์ของคลาส NGModel และ NGClone

11.2.2.2 เกมเอนจินในส่วนของอุปกรณ์อินพุต

เอนจินในส่วนของการติดต่อกับอุปกรณ์อินพุตจะเป็นการนำเอาอินเทอร์เฟซและเมทอดที่มีอยู่ในไลบรารีอินพุตมาสร้างเป็นคลาส เพื่อที่จะใช้ในการทำงานในการติดต่อสำหรับอุปกรณ์ที่เราต้องการ เช่น เมาส์, คีย์บอร์ด และจอยสติค เป็นต้น เนื่องจากไลบรารีอินพุตได้จัดการเกี่ยวกับการติดต่อกับฮาร์ดแวร์ให้แล้ว อุปกรณ์อินพุตจึงปรากฏให้เห็นในรูปของข้อมูลเท่านั้น

คุณสมบัติที่ต้องการ (Requirement)

1. สามารถทำให้ติดต่อกับข้อมูลกับอุปกรณ์มาตรฐาน เช่น เมาส์, คีย์บอร์ด และจอยสติค ได้
2. มีขั้นตอนในการใช้งานง่ายไม่ซับซ้อน

การออกแบบ

1. กำหนดวัตถุ

เนื่องจากแอปพลิเคชันของเราทำงานกับข้อมูลที่ได้รับจากไลบรารีอินพุตเท่านั้น ดังนั้นอุปกรณ์ต่างๆ ที่เชื่อมต่ออยู่จะปรากฏให้เราเห็นเป็นข้อมูลทั้งหมด ดังนั้นจึงให้ข้อมูลเป็นหลักในการค้นหาวัตถุ

ข้อมูล	วัตถุ	ระยะเวลาในการเก็บข้อมูล
อินพุต	ข้อมูลที่ได้รับจากการตรวจสอบอินพุตจากอุปกรณ์ที่ได้รับอนุญาตไว้แล้ว ไม่ว่าจะเป็นเมาส์, คีย์บอร์ด หรือจอยสติค	ตลอดเวลาที่แอปพลิเคชันทำงานอยู่

ตารางที่ 11-8 วัตถุที่ได้จากการแยกแยะโดยใช้ข้อมูลที่เก็บในส่วนของอินพุต

11.2.2.3 เกมเอนจินในส่วนของเสียงและวิดีโอ

ในส่วนของเสียงเรามีความต้องการที่แตกต่างกัน 2 รูปแบบ คือ

1. การเล่นเกมเสียงเอฟเฟ็ก เช่น เสียงฝีเท้า, เสียงประตู เป็นต้น ซึ่งการเล่นเสียงประเภทนี้มักจะต้องใช้ตำแหน่งในการคำนวณด้วย รวมทั้งยังต้องมีความสามารถในการผสมเสียงจากหลายๆ แหล่งเสียง เนื่องจากในความเป็นจริงแล้วเสียงสามารถเกิดขึ้นได้จากหลายๆ แหล่งเสียงพร้อมๆ กัน

2. การเล่นเกมเพลงหรือเสียงดนตรี เสียงเพลงหรือเสียงดนตรีจะเป็นสิ่งที่สร้างบรรยากาศให้เกิดขึ้นในเกมทำให้ผู้เล่นเกมมีอารมณ์ร่วมไปกับเหตุการณ์ที่ปรากฏขึ้น ซึ่งโดยปกติแล้วการเล่นเพลงจะเล่นเพียง 1 เพลงในแต่ละครั้งและต้องการขนาดของข้อมูลที่ไม่ใหญ่มากรวมทั้งใช้การประมวลผลไม่มาก ในสมัยก่อนไฟล์ในแบบ Midi ได้รับความนิยมมาก แต่ในปัจจุบันสามารถใช้ไฟล์ในรูปแบบที่มีขนาดเล็กและให้เสียงที่สมจริงมากกว่าได้ เช่น ไฟล์ ในรูปแบบ MP3 เป็นต้น

ในส่วนของการเล่นไฟล์วิดีโอก็สามารถใช้ไลบรารีโซว์ในการเล่นได้เช่นกัน เนื่องจากไลบรารีโซว์นั้นสามารถมีความสามารถในการเล่นไฟล์สตรีมทุกรูปแบบที่มีการสร้าง DMO (Direct Media Object) เอาไว้ (ทุกรูปแบบที่ Window Media Player สามารถเล่นได้)

การออกแบบ

1. กำหนดวัตถุ

- การเล่นเสียงเอฟเฟ็ค

วัตถุ	ข้อมูล	ระยะเวลาในการเก็บข้อมูล
เสียงเอฟเฟ็ค	ข้อมูลของเวฟที่อยู่ในรูปของไฟล์	ช่วงเวลาที่มีการเล่นเสียงเอฟเฟ็ค

ตารางที่ 11-9 วัตถุที่ได้จากการแยกแยะโดยใช้ข้อมูลที่เก็บในส่วน of เสียงเอฟเฟ็ค

- การเล่นเสียงดนตรีและเสียงเพลง

วัตถุ	ข้อมูล	ระยะเวลาในการเก็บข้อมูล
เสียงเพลงหรือเสียงดนตรี	ข้อมูลของเสียงที่ถูกบีบอัดด้วย MP3	ช่วงเวลาที่มีการเล่นเสียงเพลงหรือเสียงดนตรี

ตารางที่ 11-10 วัตถุที่ได้จากการแยกแยะโดยใช้ข้อมูลที่เก็บเสียงเพลง

- การเล่นวิดีโอ

วัตถุ	ข้อมูล	ระยะเวลาในการเก็บข้อมูล
วิดีโอ	ข้อมูลของวิดีโอที่ถูกบีบอัดด้วยอัลกอริทึมแบบต่างๆ เช่น AVI, DIVX และ MOV เป็นต้น	ช่วงเวลาที่ต้องการเล่นไฟล์วิดีโอ

ตารางที่ 11-11 วัตถุที่ได้จากการแยกแยะโดยใช้ข้อมูลที่เก็บในส่วน of วิดีโอ